

NETWORK DEFENSE SIMULATION

SOC Platform — New User / Handover Guide

Detailed project documentation for a new developer, trainee, reviewer, or SOC learner.

I developed a Network Defense Simulation and SOC platform using ReactJS, Node.js and Express.js. The objective was to understand and demonstrate the complete SOC workflow rather than only build a visual dashboard. The system generates controlled security activity, represents it as events, applies detection logic, generates alerts, allows analysts to create and investigate incidents, and provides a SOAR-style response workflow using simulated playbooks. Finally, analytics aggregates the security activity. The project is a learning-oriented implementation inspired by common SOC/SIEM concepts, not a replacement for a production SIEM.

GENERATE → DETECT → INVESTIGATE → RESPOND → ANALYZE

Purpose: Explain what the platform is, how the modules work together, how to run a complete simulation, how to investigate an alert, how SOAR fits into the workflow, and what the current prototype limitations are.

1. Project Overview

The Network Defense Simulation is a web-based, learning-oriented SOC platform. It simulates controlled network/security activity and demonstrates a SOC workflow from telemetry generation through detection, alerting, incident investigation, response, and analytics.

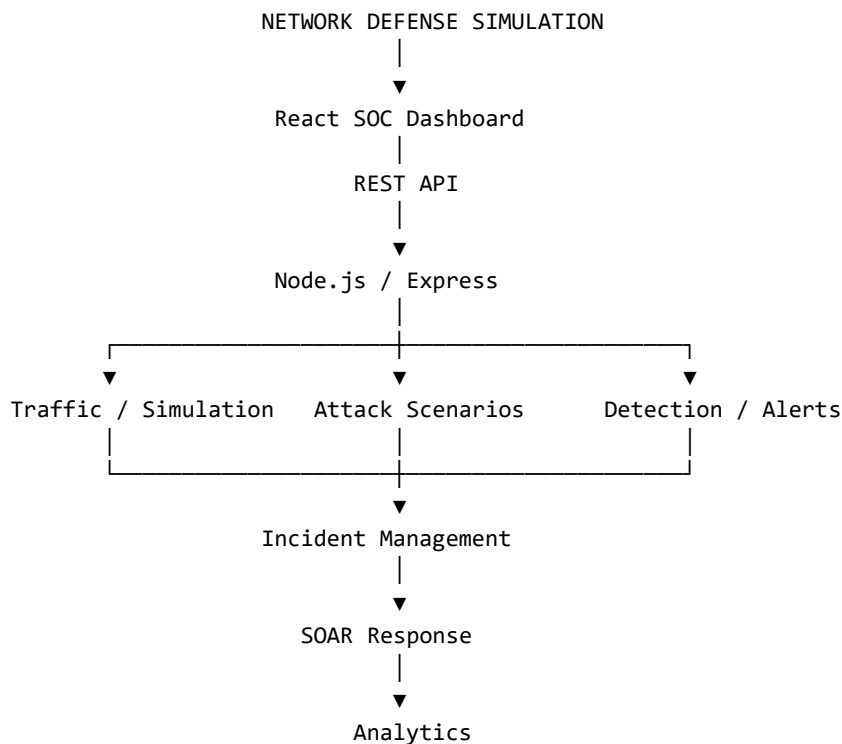
GENERATE → DETECT → INVESTIGATE → RESPOND → ANALYZE

It is not intended to replace a production SIEM such as Splunk, Microsoft Sentinel, or Wazuh. Its purpose is to help a new user understand the architecture and operational lifecycle behind security monitoring systems.

2. Project Objectives

- Generate controlled security/network activity for testing.
- Represent generated activity as events/telemetry.
- Apply detection logic and produce security alerts.
- Provide a SOC-style monitoring dashboard.
- Create and manage incidents from alerts.
- Record investigation notes and related alerts.
- Demonstrate SOAR playbooks and simulated response actions.
- Aggregate events, alerts, incidents, and response information into analytics.
- Keep backend responsibilities separated into modules.

3. High-Level Architecture



The current server implementation imports separate modules for scenario management, simulation, traffic generation, incident management, SOAR/playbooks, and analytics. This makes server.js primarily an API/orchestration layer.

4. Technology Stack

Layer	Technology	Purpose
Frontend	ReactJS + CSS	SOC dashboard, pages, components and user interaction.
Backend	Node.js + Express.js	REST APIs and coordination of security modules.
Communication	HTTP/JSON REST APIs	Frontend-to-backend communication.
Runtime state	In-memory module state	Prototype storage for generated events, alerts and incidents.
Development	VS Code / npm	Development and package management.

5. Project Structure

```
network-defense-sim/
├── client/
│   └── src/
│       ├── components/
│       ├── pages/
│       ├── api/
│       ├── App.jsx
│       ├── App.css
│       └── main.jsx
└── server/
    ├── server.js
    └── src/
        ├── engine/
        │   ├── simulationEngine.js
        │   ├── trafficEngine.js
        │   └── trafficProfiles.js
        ├── scenarios/
        │   ├── scenarioEngine.js
        │   ├── attackStages.js
        │   └── scenarioProfiles.js
        ├── detection/
        │   ├── detectionEngine.js
        │   ├── rules.js
        │   ├── riskEngine.js
        │   └── mitreMapping.js
        ├── incidents/
        │   └── incidentManager.js
        ├── response/
        │   └── playbookEngine.js
        └── analytics/
            └── analyticsEngine.js
```

6. Module Guide

Module	What it does	Main API
Traffic Generator	Creates individual simulated traffic/security events.	GET /api/traffic/test and GET /api/traffic.
Simulation Engine	Starts, stops and reports the state of the simulated environment.	POST /api/simulation/start, POST /api/simulation/stop, GET /api/simulation/status.
Scenario Engine	Runs structured attack scenarios and exposes their events and alerts.	POST /api/scenario/start, POST /api/scenario/stop, GET /api/scenario/status, /events, /alerts.
Events / Telemetry	Represents activity observed by the simulated SOC. Events are not automatically incidents.	GET /api/events combines simulation and scenario events.
Detection	Applies detection logic and provides detection statistics. Detection area also contains risk/MITRE-related modules.	GET /api/detection/stats.
Alerts	Security findings produced from simulation/scenario activity. The general alert API combines both sources.	GET /api/alerts.
Incident Management	Creates and tracks investigations, notes, related alerts, status updates and resolutions.	POST/GET/PATCH /api/incidents and related subroutes.
SOAR Response	Demonstrates Security Orchestration, Automation and Response using playbooks/actions. Current response is simulated.	GET /api/response/playbooks, GET /api/response/actions, POST /api/response/execute.
Analytics	Aggregates events, alerts, incidents and response information.	GET /api/analytics.
Authentication capability	Current server contains login, current-user and ADMIN user-list routes, but authentication is not consistently applied to every other API route.	POST /api/auth/login, GET /api/auth/me, GET /api/auth/users.

7. Core Concepts for a New User

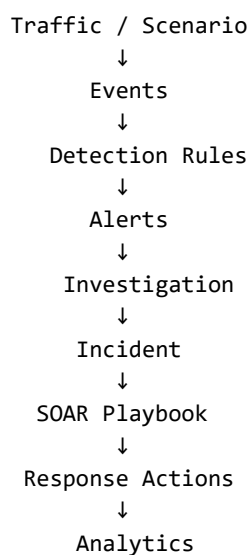
Concept	Meaning	Example
Event	A simulated activity/telemetry record.	A login attempt occurs.
Detection	Logic that checks activity for suspicious conditions.	A repeated-failure pattern matches a rule.

Alert	A security finding requiring attention.	A suspicious authentication alert appears.
Incident	An investigation case built around an alert or related alerts.	INC-001 contains the investigation.
Playbook	A predefined response workflow.	Investigate → contain → document.
Response action	A step executed by a playbook.	Simulate a source-blocking action.
Analytics	Aggregated security information.	Event, alert and incident counts.

Important: Event ≠ Alert ≠ Incident. A new user should understand this before using the dashboard.

8. End-to-End SOC Workflow

1. Start the backend API and frontend.
2. Start the simulation or an attack scenario.
3. Observe generated events in Events.
4. Wait for or trigger detection activity.
5. Review the resulting security alert.
6. Create an incident from the selected alert.
7. Open the incident and document investigation findings.
8. Link related alerts when they belong to the same investigation.
9. Use SOAR to inspect playbooks/actions and execute a simulated response.
10. Open Analytics and review aggregated results.
11. Stop the simulation after testing.



9. New User Setup

9.1 Prerequisites

- Node.js and npm installed.
- Modern browser.
- VS Code or another code editor.
- Project folder available locally.
- Docker is not required for the current setup.

9.2 Start Backend

```
cd C:\Users\ADMIN\network-defense-sim\server
npm install
node server.js
```

Expected message:

```
Network Defense API running on http://localhost:5000
```

9.3 Start Frontend

```
cd C:\Users\ADMIN\network-defense-sim\client
npm install
npm run dev
```

Open the Vite URL shown in the terminal.

9.4 Verify Backend

```
http://localhost:5000/
```

Expected JSON: { "message": "Network Defense Simulation API", "status": "online" }

10. API Reference

Method	Endpoint	Purpose
GET	/	Backend health check.
GET	/api/traffic/test	Generate one traffic test event.
GET	/api/traffic	Retrieve traffic events.
POST	/api/simulation/start	Start simulation.
POST	/api/simulation/stop	Stop simulation.
GET	/api/simulation/status	Simulation status.
GET	/api/alerts	Combined simulation + scenario alerts.
GET	/api/events	Combined simulation + scenario events.
GET	/api/detection/stats	Event, alert and detection-rule statistics.
POST	/api/scenario/start	Start an attack scenario.
POST	/api/scenario/stop	Stop an attack scenario.
GET	/api/scenario/status	Scenario status.

GET	/api/scenario/events	Scenario events.
GET	/api/scenario/alerts	Scenario alerts.
POST	/api/incidents	Create incident from alertId.
GET	/api/incidents	List incidents.
GET	/api/incidents/:id	Get one incident.
PATCH	/api/incidents/:id	Update incident.
POST	/api/incidents/:id/notes	Add investigation note.
POST	/api/incidents/:id/alerts	Link an alert.
POST	/api/incidents/:id/resolve	Resolve incident.
GET	/api/response/playbooks	List playbooks.
GET	/api/response/actions	List response actions.
POST	/api/response/execute	Execute/simulate response for an alert.
GET	/api/analytics	Build analytics.
POST	/api/auth/login	Login.
GET	/api/auth/me	Current authenticated user.
GET	/api/auth/users	ADMIN-only user list in current implementation.

11. Example Investigation Story

Example: suspicious repeated authentication activity

1. Simulation/scenario generates authentication-related events.
2. Events are exposed to the monitoring layer.
3. Detection logic identifies a suspicious pattern.
4. An alert is produced.
5. Analyst reviews the alert.
6. Analyst creates an incident.
7. Analyst records evidence/investigation notes.
8. Related alerts can be attached to the incident.
9. A SOAR playbook can be executed in simulation.
10. Analytics summarizes the activity.

Use the actual generated values during a live demo; this story explains the workflow rather than promising fixed alert values.

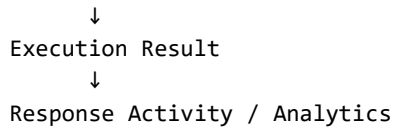
12. SOAR Explanation for New Users

SOAR means Security Orchestration, Automation and Response. In this project it is represented by playbooks and response actions. The same alert that an analyst investigates can be passed to the response layer.

```

Security Alert
  ↓
Select alert
  ↓
SOAR Playbook
  ↓
Response Actions

```



The current implementation should be described as simulated response. It should not be represented as actually blocking an attacker, disabling a user, or changing production infrastructure unless a real integration has been intentionally added.

13. Troubleshooting

Problem	Likely cause	Check
Dashboard blank	React runtime/routing error.	Browser console, App.jsx, imports, routes.
Unable to load incidents	Backend/API problem.	Check backend, /api/incidents, frontend API base URL.
No alerts	No matching simulation/scenario activity yet.	Start simulation/scenario; inspect /api/events and /api/alerts.
SOAR shows playbooks but no actions	Frontend is not displaying or executing response-action data.	Check /api/response/actions and SOAR page integration.
Alert not found	Alert ID is stale/not currently in alert collections.	Refresh alerts and use the current ID.
localhost:5000 unavailable	Backend stopped or startup error.	Run node server.js and inspect terminal.
Data disappears after restart	Runtime state is in memory.	Restart simulation and regenerate test data.
npm.ps1 blocked	PowerShell execution policy issue.	Use an allowed terminal/configuration according to local policy.

14. Current Limitations

- The project is a simulation/learning environment, not a production SIEM.
- Security activity is simulated rather than collected from a real enterprise network.
- SOAR response is simulated rather than connected to real infrastructure controls.
- The current prototype uses in-memory runtime state; restarting the backend can clear generated data.
- Authentication capability exists in the current server file, but sensitive API protection is not yet consistently applied across all modules.
- Detection logic and threat-intelligence coverage are intentionally smaller than enterprise platforms.

15. Security and Production Considerations

- Apply authentication and authorization consistently to every sensitive endpoint.
- Use secure secret management and HTTPS.
- Add persistent storage and backups.

- Add audit logging for analyst actions.
- Validate API input and implement rate limiting.
- Persist SOAR execution history.
- Separate simulation/test integrations from production infrastructure.
- Add automated unit, integration, and security tests before production deployment.

16. Testing Checklist

Test	Expected result
Backend health	GET / returns online status.
Traffic test	A traffic event is returned.
Simulation start/stop	Status changes correctly.
Events	Generated events appear.
Alerts	Relevant detection/scenario alerts appear.
Detection stats	Counts and rule statistics are returned.
Scenario	Scenario starts/stops and produces scenario data.
Incident creation	Valid alertId creates an incident.
Incident note	Note is added to the incident.
Related alert	Alert can be linked.
Incident resolution	Resolution is recorded.
SOAR playbooks	Playbooks are listed.
SOAR actions	Actions are returned.
SOAR execution	Selected alert produces response results.
Analytics	Aggregated analytics are returned.

17. Future Roadmap

- Persistent MongoDB/PostgreSQL storage.
- Real-time WebSocket/SSE event updates.
- More configurable detection rules.
- Expanded MITRE ATT&CK mapping.
- Threat-intelligence enrichment.
- Complete RBAC/authentication across all sensitive APIs.
- Audit logs for analyst actions.
- Persistent SOAR execution history.
- Advanced threat hunting and filtering.
- More detailed analytics and reports.
- Automated backend/frontend tests.
- Controlled integrations with external security platforms in a dedicated test environment.

18. New User Learning Path

- Learn event vs alert vs incident.
- Start backend and frontend.
- Verify the health endpoint.
- Generate a traffic test event.
- Start the simulation and watch Events.
- Run an attack scenario and compare scenario events with normal activity.
- Inspect Security Alerts.
- Review detection statistics.
- Create and investigate an incident.
- Add notes and related alerts.
- Resolve the incident.
- Run a SOAR playbook against an alert.
- Review the response result.
- Review Analytics.
- Repeat with another scenario.

19. Glossary

Term	Definition
SOC	Security Operations Center; security monitoring, investigation and response function.
SIEM	Security Information and Event Management.
SOAR	Security Orchestration, Automation and Response.
Telemetry	Data generated by monitored systems.
Detection rule	Logic that identifies suspicious activity.
Alert	Security finding requiring attention.
Incident	Tracked security investigation/case.
Playbook	Predefined response workflow.
MITRE ATT&CK	Knowledge base used to describe adversary tactics and techniques.
REST API	HTTP/JSON interface between frontend and backend.
In-memory state	Runtime data that is not persisted to a database.

20. Quick Command Appendix

Backend

```
cd C:\Users\ADMIN\network-defense-sim\server  
npm install  
node server.js
```

Frontend

```
cd C:\Users\ADMIN\network-defense-sim\client  
npm install  
npm run dev
```

Health

```
http://localhost:5000/
```

Useful endpoints

```
GET /api/traffic  
GET /api/events  
GET /api/alerts  
GET /api/detection/stats  
GET /api/incidents  
GET /api/response/playbooks  
GET /api/response/actions  
GET /api/analytics
```